EPFL - Computer Science

COM-407 Advanced Networks

Lecture 09 — 13/11/2025

Author: Jonathan Dutoit

Professor: Pavlos Nikolopoulos

Routing Algorithms

Contents

1. Routing in General
2. Link state routing, OSPF Single Area
3. Dijkstra's algorithm
4. Equal Cost Multipath
5. Topology Changes
6. Security of OSPF
7. OSPF, multiple areas
8. Other uses of Link State
9. Software Defined Networking (SDN)

Reminder: the network layers

A router has 2 functions:

Forwarding: When a packet arrives at a router's input link, the routers moves the packet to the appropriate outgoing link. This is based on:

- Information in the packet header (e.g., dest IP address, BIER extension header, etc.)
- the router's *forwarding table* (a.k.a *routing table*)

Routing: Determines the entire route/path followed by each packet towards its destination hence the entries of the forwarding tables and/or information in extension header. This is typically done by a *routing algorithm*

Routing protocols (or routing algorithms)

Why we need them?

- Manually configuring the forwarding tables is not easy. In a single domain, it's doable but time-consuming and error-prone. In the WAN. it is impossible - about 120000 domains should coordinate.
- A routing protocol or algorithm thus allows routers to automatically compute the best path(s) to each destination.

How they differ?

Optimization objective of the algorithm

- To use least-cost path
- To use equal-cost multi-path
- To respect policies
- Arbitrary (min delay, low bandwidth usage...)

The underlying network, topology available?

- Single domain ⇒ infra-domain routing (main alg. is OSPF)
- Multiple domains ⇒ inter-domain routing (main alg. is BGP)

Where is the output stored?

- Inside a local forwarding table
- Directly into the packet headers

Taxonomy of routing protocols

Link State: Each router maintains a local *map* of the entire topology

- Obtained by *gossiping* with other routers (flooding)
- Every link on map has a cost; e.g. cost (1 Gb/s link) = 1

Furthermore, it computes the shortest (min-cost) paths to each destination prefix based on map and determines next hop to each prefix and populates its forwarding table.

It's mainly used for *intra-domain routing* (e.g., OSPF, IS-IS) and advanced bridging methods (e.g., TRILL, SPB Shortest Path Bridging)

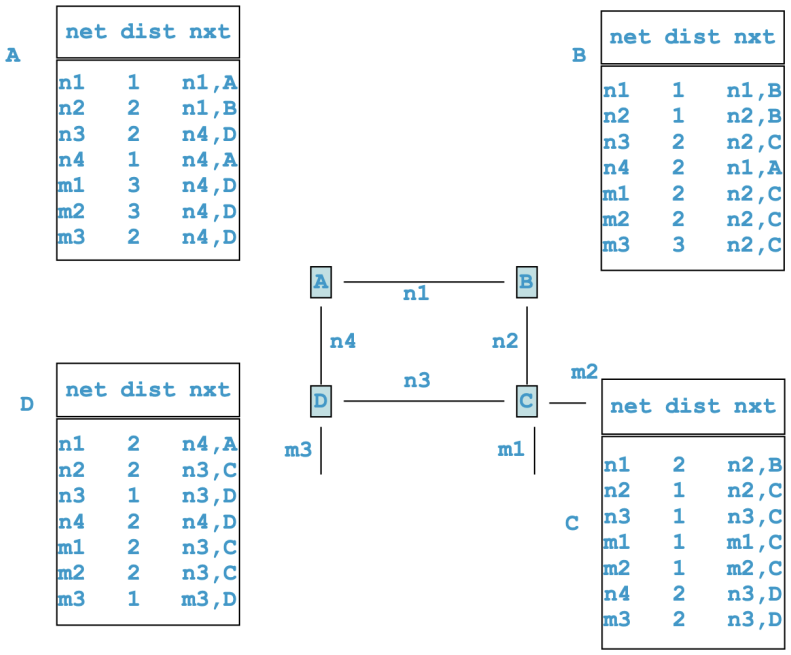
Distance Vector: *No* global map. Each router initially knows only about:

- Locally-attached networks and neighbour routers,
- And the costs of direct links to them

Then, it ***informs*** neighbors about the estimated distances to all destinations it knows (= sends its distance vector); ***learns*** new destinations from vectors received by neighbors and ***updates*** its distance vector (using the Bellman-Ford algorithm).

Finally: it determines next hops and populates its forwarding table.

- All link costs = 1
- This message-passing method *converges* to shortest paths!
- But, it may need *time to converge*
- Hence it is used in *small* networks



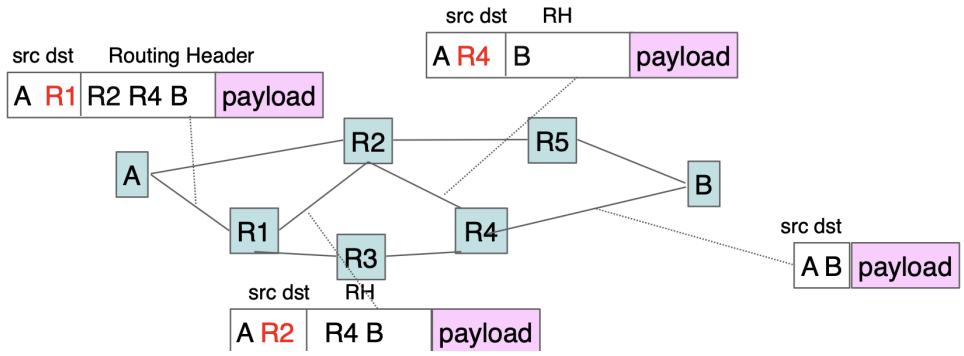
Path Vector: We don't know the topology nor the costs. Hence, every router knows only:

- Its neighbors plus explicit *paths* to all destinations

It then computes the "best" path to each destination and populates forwarding table. Optimization criterion is *not cost*.

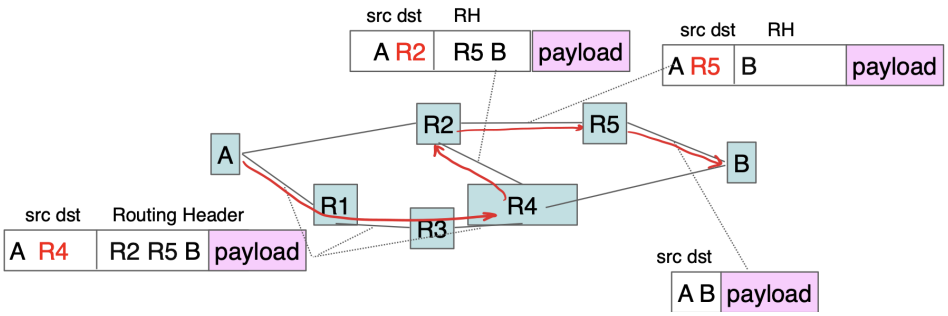
This is suitable for *inter-domain routing*, because it is hard to assign costs and domains may do not want to disclose their private internal topology. Computation may be based on the amount of hops for example.

- e.g., BGP computes domain-level paths
- Each router selects best path according to *multiple criteria*



Source Routing "Strict"

- Source puts explicit paths *into packet headers* (path = sequence of intermediate hops)
- In IPv6, routing header is an extension header - contains intermediate hops and ultimate destination; destination IP address is next intermediate hop
- Intermediate routers are "dumb": just remove themselves from header and forward packets to next hop
- Used in *ad-hoc networks*, where route computation is done by a control application or by discovery: e.g.: source discovers path by flooding *explorer packets* that accumulate the path followed



Source Routing "Loose"

- Forces *some* intermediate hops
- Assumes an *underlying* routing algorithm such as link-state routing, e.g., we know how to go from A to R4
- Used for fine grained control (traffic engineering, separation of costumers)

Segment Routing

- Generalizes loose source routing by allowing the routing header to contain processing instructions for the intermediate hop: e.g., we can ask R4 to apply a security *function* (screening, traffic separation)
- Used in data centers

2. OSPF (for a single area)

Most important link state algorithm (*"Each router maintains a local map of the entire topology"*). Every router has:

- an *interface database* (= its network interfaces, learnt by configuration)
- an *adjacency database* (= neighbors' states, learnt by a hello protocol)
- a *link state database* (= topology map, learnt by flooding)

Hello is a ***ping-style*** protocol between immediate neighbors

- discovers neighbor routers
- detects failures (if neighbor doesn't respond after 3 times, it is considered "dead")

Link State Database and LSAs

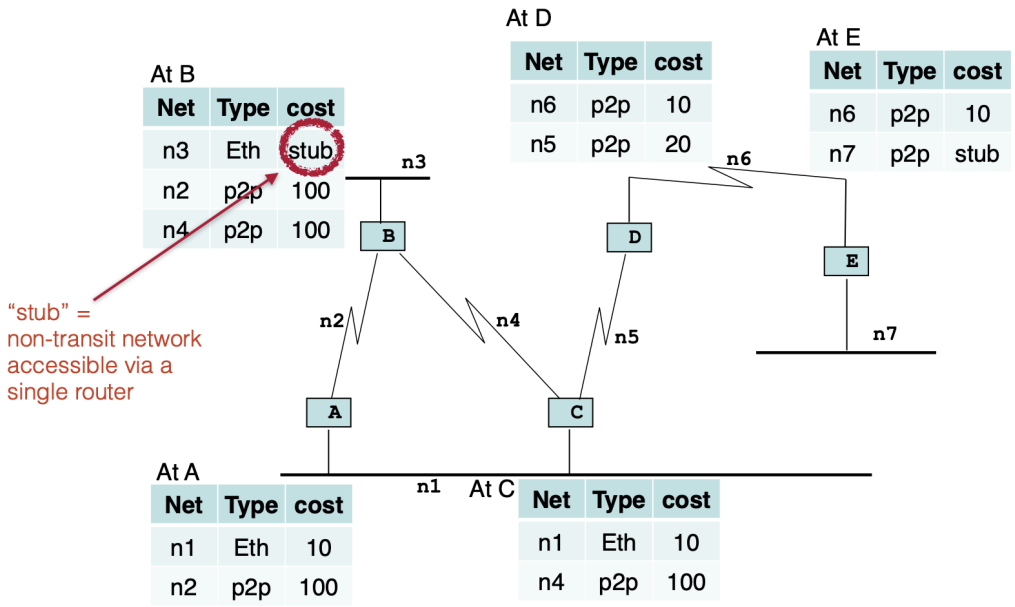
When two routers become neighbors, they first *synchronize* their link state databases:

- **Case 1**: one router is new, hence *copies* what the other already knows
- **Case 2:** existing routers connect, hence they *merge* their database

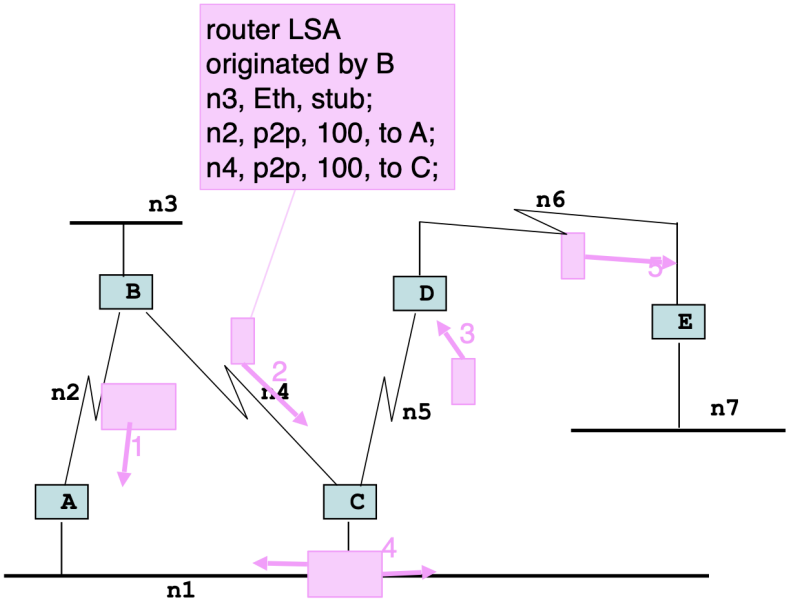
Once synchronized, a router sends and accepts *link state advertisements (LSAs):*

- Every router sends a LSA with its attached networks and neighbor routers
- LSAs are *flooded* in the entire area and stored in each router's link state database
- LSAs contains a *sequence* number and *age*. Sequence number prevents loops since only messages with new sequence numbers are accepted and re-flooded; Age field is used to periodically resend LSA and to flush invalid LSAs

Example showing interface databases

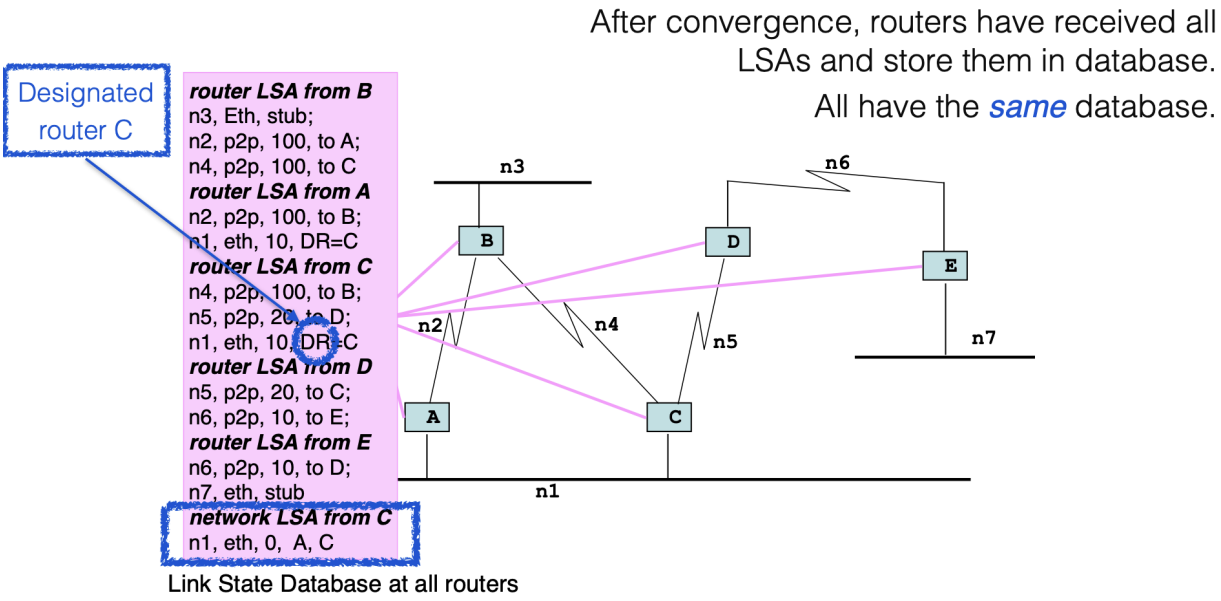


Routers flood their LSAs throughout area



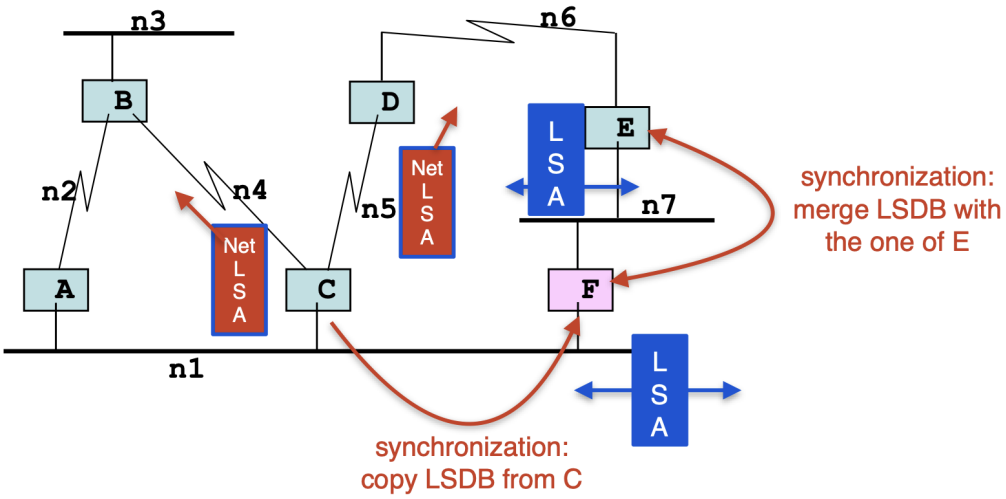
- The LSA describes all the networks attached to B and their costs, as well as the adjacent routers
- A "stub" network means non transit, i.e. there is no other router on this network. A stub network can be reached by only a single router; so, all we need to know is how to reach this router. No need to allocate a cost to a stub network.

After flooding



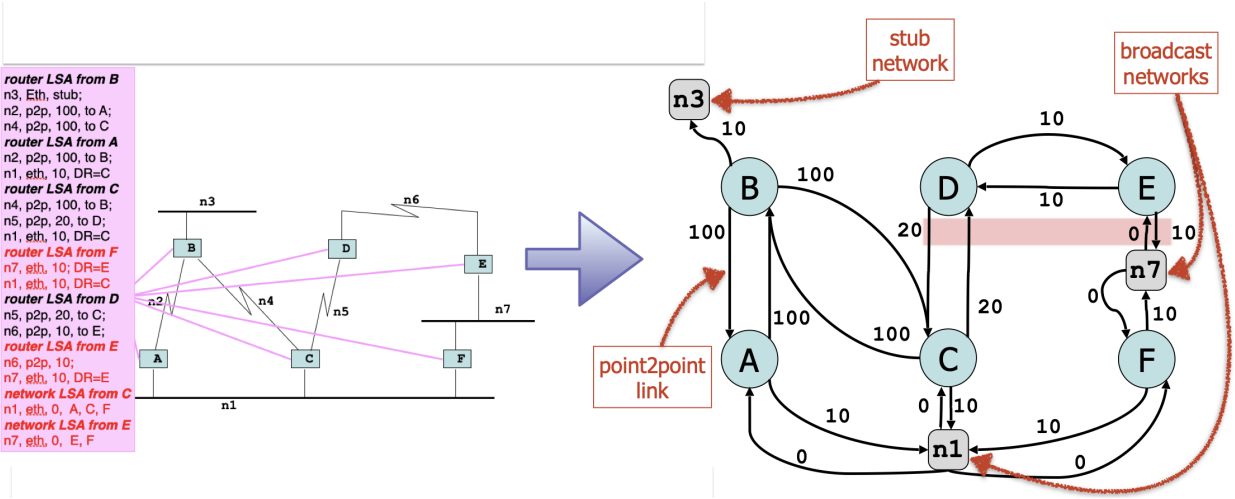
- In order to avoid issues as high number of OSPF packets that are sent and processed by each router as well as redundancy in case of failures, the routers elect a designated router per LAN (and a backup designated router)
- The designated router is typically the oldest running router and "speaks for the switch" and sends a "network LSA" which gives the list of all routers connected to the LAN.

New Router Boots



- Synchronization steps: boot up, identify neighbors, check designated router and copy their database
- After that, F starting issues new LSAs. C will also issue new LSAs saying "F is new member of this network"
- Finally, establish adjacency with other network, and you do the same thing with E (merging own views of topologies). E will send its new version of the LSAs

From LSDB to Topology Graph



Practical Aspects

- OSPF packets are sent *directly over IP*
- Reliable transmission is managed by OSPF using **acknowledgements** and **timers** (like the *stop and go* protocol)
- Routers IDs or Area IDs are 32 bit numbers

3. Shortest paths are found with Dijkstra's Algorithm

Each router runs Dijkstra *independently* on its local LSDB copy. *Why?*

- every router computes *shortest paths starting from itself*
- LSAs guarantee *no persistent* inconsistencies (LSDBs are mostly identicaly)

Also, each router computes *one or several* shortest paths to every other node

Dijkstra's Shortest Path Algorithm

`m(0) = 0; m(i) =` ∞ $\forall$ $i \neq 0$; $V = \emptyset$; `pred(i) =` $\emptyset$ $\forall i$;

```
for k = 0:N do
  find i not in V that minimizes m(i)
  if m(i) is finite
    add i to V
    for all neighbors j not in V of i
      if m(i) + c(i, j) < m(j)
        m(j) = m(i) + c(i, j)
        pred(j) = [i]
      else if m(i) + c(i, j) = m(j)
        m(j) = m(i) + c(i, j)
        pred(j) = pred(j) U [i]
```

This pseudo-code builds the shortest path tree from this node to all other nodes.

The line `find i not in V that minimizes m(i)` adds *one node at a time* to the set $\boldsymbol{V}$ of visited nodes, by picking the node that is ***closest*** to the source in terms of least cost path

Note: There are multiple version of Dijkstra's algorithm. The version presented above finds all shortest paths, other version find only one shortest path to every destination. The version presented is very close to what is really implemented in OSPF (with a difference, next-hop versus pred())

The worst-case complexity of this version is $\boldsymbol{O(N^2)}$ where N is the number of nodes. More efficient versions of the algorithm have a smaller complexity $O(NlogN + E)$ where E is the number of links.

Path Computation

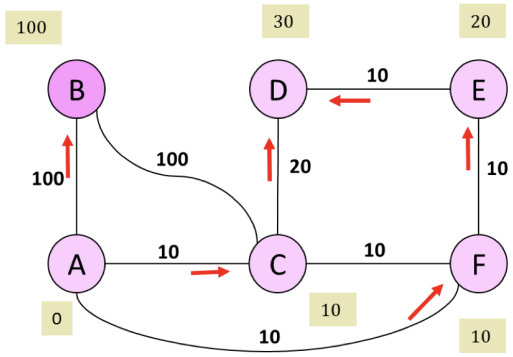
- `pred(i)` is the set of predecessors of node i on all shortest paths from source to i
- Shortest paths can be computed *backwards*, using pred(), starting *from destination*

From shortest paths to Forwarding Table

Router A keeps in its forwarding table only the *next-hop* ad the *distance* to every destination (not the entire path):

At A

Dest	Next-hop	cost
B	B	100
C	C	10
D	C or F	30
E	F	20
F	F	10



Bringing back the Network Nodes

In practice OSPF adds to the graph the network nodes, which makes the graph ***larger***

To optimize computation:

- First, stub networks *are removed*
- Then, Dijkstra is run and the forwarding table contains costs and next hop to edge routers
- Finally, stub networks are *added to the forwarding table* one by one, using information about edge routers.

4. Equal Cost Multipath

OSPF supports multiple shortest paths. IP allows to have multiple next-hops to the same destination in the forwarding table

- good: it exploits the *redundancy* of paths
- bad: the number of multiple paths may be *large* so, typically, we use a ***limit*** of multiple paths

What should router A do when it has several packets to send to destination D?
A. Send them to next-hop F or C randomly with equal probability
B. Choose one next-hop and send all packets to this next-hop
C. Test the availability of the next-hop before sending
D. Something else
E. I don't know

Answer: A and D. It's better to use **all** available paths network (load balancing) ⇒ send to all next-hops with equal probability.

However, this may cause *packet re-ordering* ⇒ alternative approach: *per-flow load balancing* requires that packets of the same flow to be sent to the same next-hop.

Text